



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Client-State Manipulation

Agenda



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Web application – a group of programs used by the server to respond to client (browser) requests.
 - Often accept input from their users: Please don't trust, should validate!
- HTTP, is stateless, servers would not keep state
 - Web server applications use state to process transactions.
 - State info may be sent to client who echoes it back in later requests
- “Hidden” values in HTML are not really hidden, still be able to manipulated

Pizza Delivery Web Site Example



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Web app for delivering pizza
 - Order: order.html – say user buys one pizza @ \$5.50
 - Confirmation: generated by confirm_order script, asks user to verify purchase, price is sent as hidden form field
 - Fulfillment: submit_order script handles user's order received as GET request from confirmation form
(variables *pay* and *price* are retrieved from URL Parameters)

Pizza Web Site Code



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Confirmation Form:

```
Exp1.html
1 <HTML>
2 <HEAD>
3 <TITLE>Pay for Pizza</TITLE>
4 </HEAD>
5 <BODY>
6 <FORM ACTION="submit_order" METHOD="GET">
7 The total cost is 5.50.
8 Are you sure you would like to order?
9 <INPUT TYPE="hidden" NAME="price" VALUE="5.50">
10 <INPUT TYPE="submit" NAME="pay" VALUE="yes">
11 <INPUT TYPE="submit" NAME="pay" VALUE="no">
12 </BODY>
13 </HTML>
```

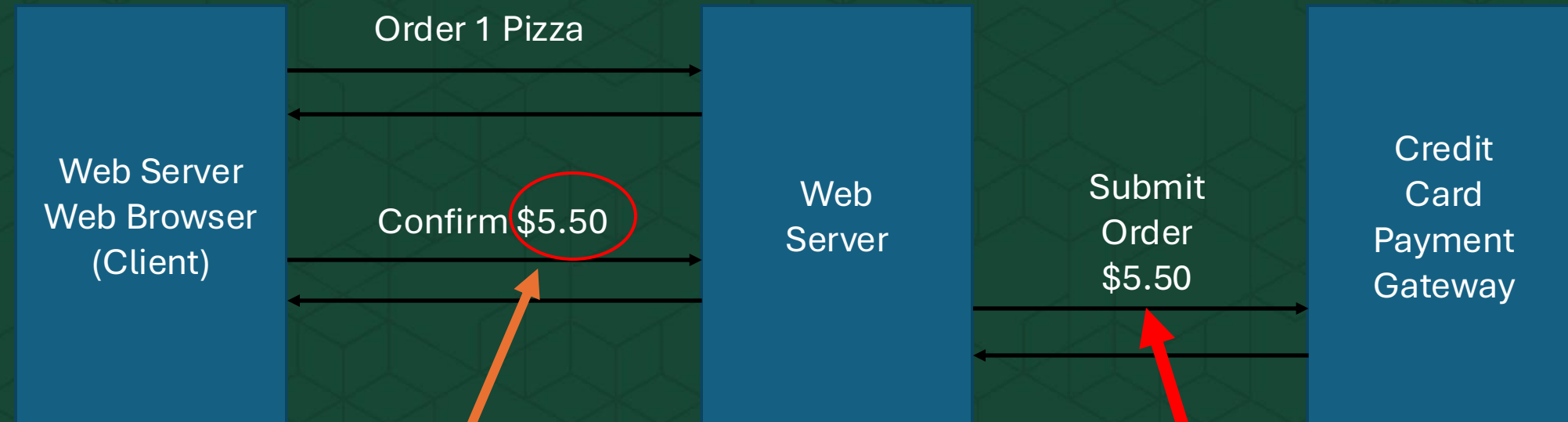
Submit Order Script:

```
Exp1.html submit.js
1 if (pay = yes) {
2 success = authorize_credit_card_charge(price);
3     if (success) {
4         settle_transaction(price);
5         dispatch_delivery_person();
6     }
7     else {
8         // Could not authorize card
9         tell_user_card_declined();
10    }
11 }
12 else {
13     // pay = no
14     display_transaction_cancelled_page();
15 }
```

Buying Pizza Example



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



Price Stored in Hidden Form Variable
submit_order?price=5.50

Attacker will modify

Attack Scenario

- Attacker navigates to order form..



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

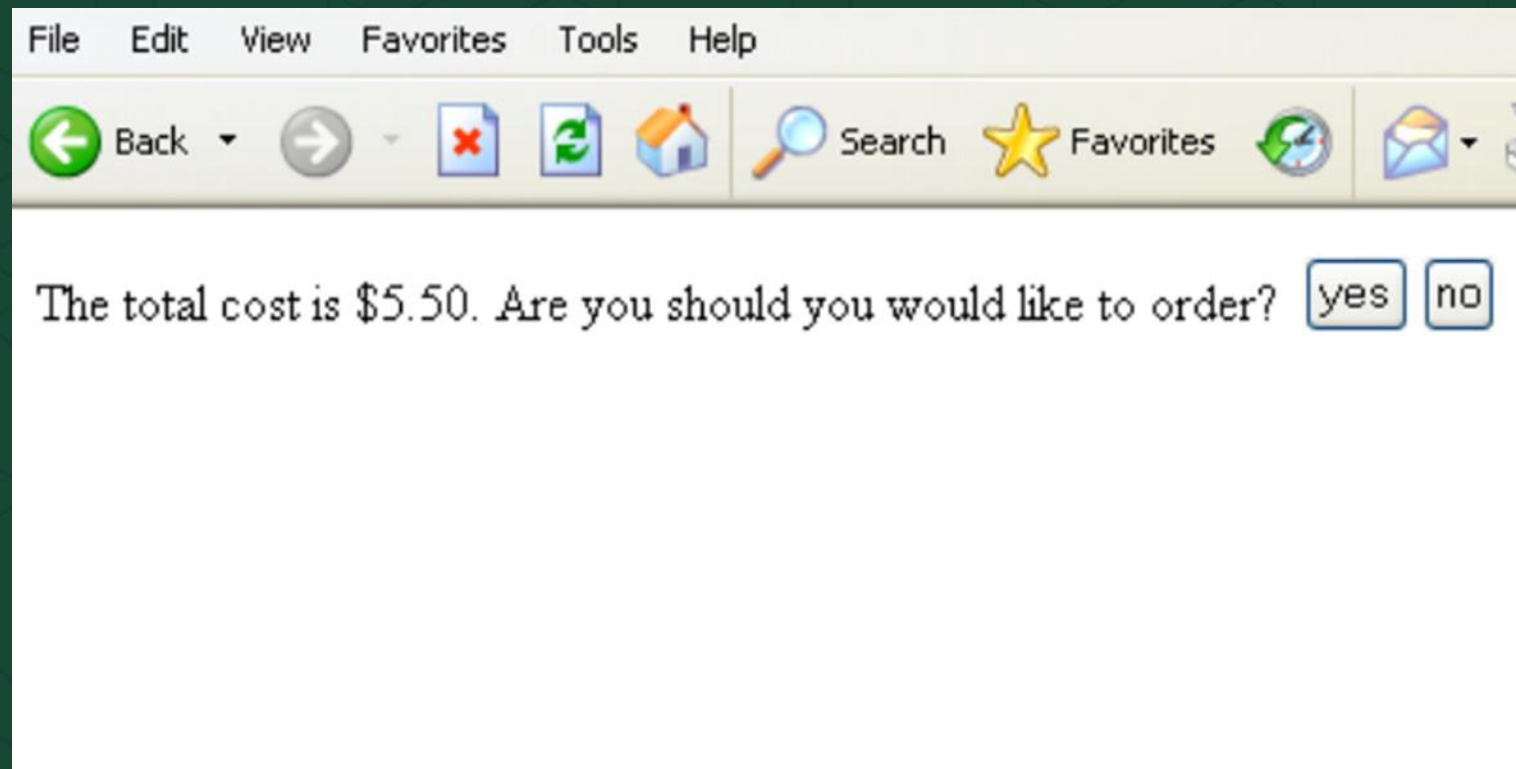
A screenshot of a web browser window. The browser's menu bar includes 'File', 'Edit', 'View', 'Favorites', 'Tools', and 'Help'. Below the menu bar is a toolbar with icons for 'Back', 'Forward', 'Stop', 'Reload', 'Home', 'Search', and 'Favorites'. The main content area of the browser displays a form with the text 'How many pizzas would you like to order?' followed by a text input field containing the number '1'. Below this is a label 'Credit Card No' followed by a text input field. To the right of the 'Credit Card No' field is a button labeled 'Order'.

Attack Scenario

- ...then to submit order form



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



Attack Scenario

- Attacker can View Page Source | Save As:



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

```
Exp1.html
1  <HTML>
2  <HEAD>
3  <TITLE>Pay for Pizza</TITLE>
4  </HEAD>
5  <BODY>
6  <FORM ACTION="submit_order" METHOD="GET">
7  The total cost is 5.50.
8  Are you sure you would like to order?
9  <INPUT TYPE="hidden" NAME="price" VALUE="5.50">
10 <INPUT TYPE="submit" NAME="pay" VALUE="yes">
11 <INPUT TYPE="submit" NAME="pay" VALUE="no">
12 </BODY>
13 </HTML>
```

Attack Scenario



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- 1. Changes price in source, reloads page locally!

- 2. Browser sends request:
 - GET /submit_order?price=0.01&pay=yes HTTP/1.0

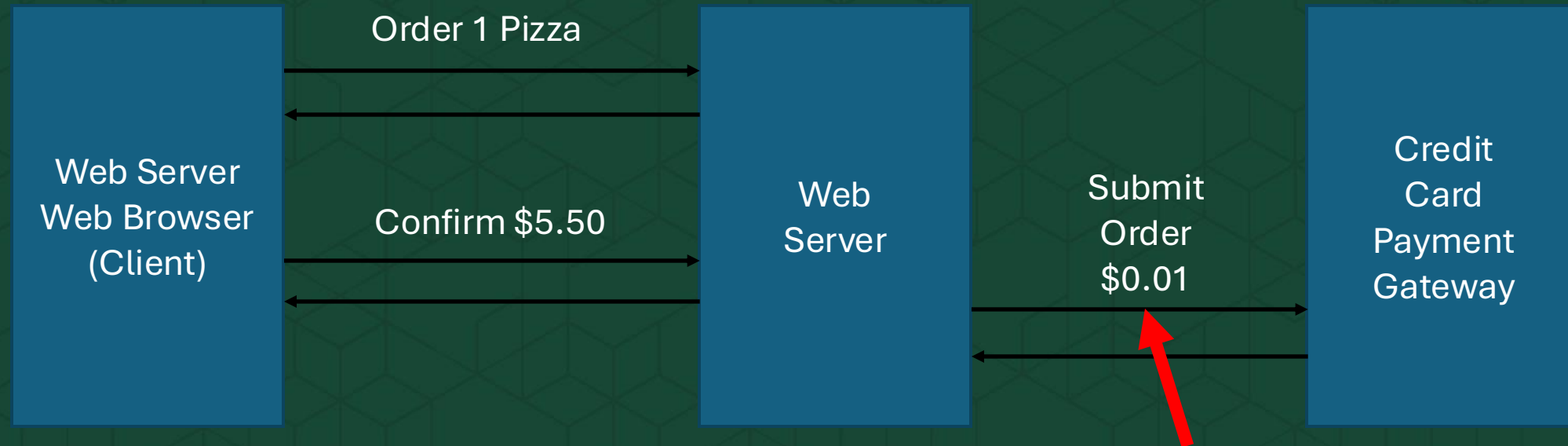
- 3. Hidden form variables are essentially in clear

```
1 <HTML>
2 <HEAD>
3 <TITLE>Pay for Pizza</TITLE>
4 </HEAD>
5 <BODY>
6 <FORM ACTION="submit_order" METHOD="GET">
7 The total cost is 5.50.
8 Are you sure you would like to order?
9 <INPUT TYPE="hidden" NAME="price" VALUE="0.01">
10 <INPUT TYPE="submit" NAME="pay" VALUE="yes">
11 <INPUT TYPE="submit" NAME="pay" VALUE="no">
12 </BODY>
13 </HTML>
```

Attack Scenario



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —



Attacker modify Price!

Attack Scenario



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Command-line tools to generate HTTP requests
- curl or Wget automates & speeds up attack
 - `curl https://www.deliver-me-pizza.com/submit_order?price=0.01&pay=yes`
- The attacker can specify parameters as arguments to curl or Wget command
 - `curl -dprice=0.01 -dpay=yes https://www.deliver-mepizza.com/submit_order`
 - `wget --post-data 'price=0.01&pay=yes' https://www.deliver-me-pizza.com/submit_order`

Solution 1: Authoritative State Stays at Server



- Server sends session-id to client
 - Server has table mapping session-ids to prices
 - Randomly generated (hard to guess) 128-bit id sent in hidden form field instead of the price.
 - `<INPUT TYPE="hidden" NAME="session-id" VALUE="3927a837e947df203784d309c8372b8e">`
- New Request
 - `GET /submit_order?session-id=3927a837e947df203784d309c8372b8e&pay=yes HTTP/1.0`

Solution 1 Changes

- submit_order script changes:

```
1  <script type="text/javascript">
2  if (pay=yes) {
3      price = lookup(session-id);
4      if (price!=null) {
5          //same as before
6          ...
7      }
8      else{//cannot find session
9          display_transaction_cancelled_page();
10         log_client_IP_and_info();
11     }
12
13 }else{
14     //same no case
15
16 }
17 </script>
18
```

Session Management



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- 128-bit session-id, $n = \#$ of session-ids
 - Limit chance of correct guess to $n/2^{128}$
 - Time-out idle session-ids
 - Clear expired session-ids
 - Session-id: hash random # & IP address – harder to attack (also need to spoof IP)
- Con: server requires DB lookup for each request
 - Performance bottleneck – possible DoS from attackers sending random session-ids

Solution 2: Signed State Sent to Client



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY

- Keep Server stateless, attach a signature to state and send to client
 - detect tampering through MACs (Message Authentication Codes)
 - Sign whole transaction (based on all parameters)
 - Security based on secret key known only to server

```
Exp1.html • Changes price.html x submit.js x submit_order script changes.js x ned State Sent to Client.html x
1 <HTML>
2 <HEAD>
3 <TITLE>Pay for Pizza</TITLE>
4 </HEAD>
5 <BODY>
6 <FORM ACTION="submit_order" METHOD="GET">
7 The total cost is 5.50.
8 Are you sure you would like to order?
9 <INPUT TYPE="hidden" NAME="item-id" VALUE="1384634">
10 <INPUT TYPE="hidden" NAME="qty" VALUE="1">
11 <INPUT TYPE="hidden" NAME="address"
12 VALUE="123 Main St, Stanford, CA">
13 <INPUT TYPE="hidden" NAME="credit_card_no"
14 VALUE="5555 1234 4321 9876">
15 <INPUT TYPE="hidden" NAME="exp_date" VALUE="1/2012">
16 <INPUT TYPE="hidden" NAME="price" VALUE="5.50">
17 <INPUT TYPE="hidden" NAME="signature"
18 VALUE="a2a30984f302c843284e9372438b33d2">
19 <INPUT TYPE="submit" NAME="pay" VALUE="yes">
20 <INPUT TYPE="submit" NAME="pay" VALUE="no">
21 </BODY>
22 </HTML>
```

S&T

Solution 2 Analysis

- Changes in submit_order script:
 - Can detect tampered state vars from invalid signature
- Performance Hit
 - Compute MACs when processing HTTP requests
 - Stream state info to client -> extra bandwidth

```
Exp1.html • Changes price.html x submit.js x submit_order_script changes.js x rtion2 submit_order_script.js x
<script type="text/javascript">
1 if (pay = yes) {
2 // Aggregate transaction state parameters
3 // Note: | is the concatenation operator
4 // and # is a delimiter.
5 state = item-id | # | qty | # | address | # |
6 credit_card_no | # | exp_date | # | price;
7
8 // Compute message authentication code with
9 // server key K.
10 signature_check = MAC(K, state);
11 if (signature == signature_check) {
12 success = authorize_credit_card_charge(price);
13 if (success) {
14 settle_transaction(price);
15 dispatch_delivery_person();
16 }
17 else {
18 // Could not authorize card
19 tell_user_card_declined();
20 }
21 }
22 else {
23 // Invalid signature
24 display_transaction_cancelled_page();
25 log_client_IP_and_info();
26 }
27 }
28 else {
29 display_transaction_cancelled_page();
30 }
</script>}
```

POST v.s. GET

- GET and POST are two common HTTP methods.
- An HTTP request with GET command is used for requesting data from a server
- An HTTP request with POST command is used for sending data to a server to create/update a resource.



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Security vulnerabilities of the GET method



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

```
Exp1.html • Changes price.html x submit.js x submit_order script changes.js x ned State Sent to Client.html x
1  <HTML>
2  <HEAD>
3  <TITLE>Pay for Pizza</TITLE>
4  </HEAD>
5  <BODY>
6  <FORM ACTION="submit_order" METHOD="GET">
7  The total cost is 5.50.
8  Are you sure you would like to order?
9  <INPUT TYPE="hidden" NAME="item-id" VALUE="1384634">
10 <INPUT TYPE="hidden" NAME="qty" VALUE="1">
11 <INPUT TYPE="hidden" NAME="address"
12 VALUE="123 Main St, Stanford, CA">
13 <INPUT TYPE="hidden" NAME="credit_card_no"
14 VALUE="5555 1234 4321 9876">
15 <INPUT TYPE="hidden" NAME="exp_date" VALUE="1/2012">
16 <INPUT TYPE="hidden" NAME="price" VALUE="5.50">
17 <INPUT TYPE="hidden" NAME="signature"
18 VALUE="a2a30984f302c843284e9372438b33d2">
19 <INPUT TYPE="submit" NAME="pay" VALUE="yes">
20 <INPUT TYPE="submit" NAME="pay" VALUE="no">
21 </BODY>
22 </HTML>
```

Security vulnerabilities of the GET method



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

- Alice wants to buy one pizza from www.deliver-me-pizza.com
- Alice enters the number of pizza, credit card number, and clicks the button “order”.

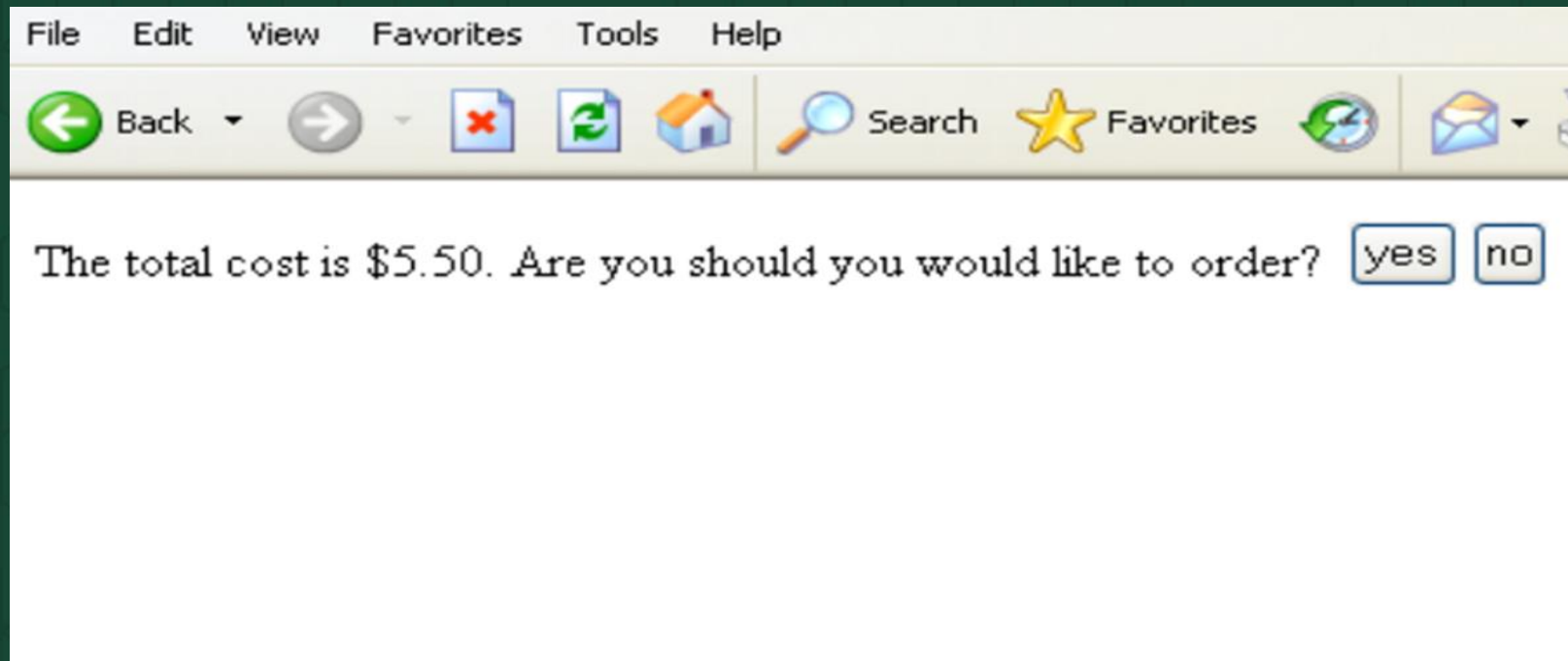
A screenshot of a web browser window. The browser's menu bar includes 'File', 'Edit', 'View', 'Favorites', 'Tools', and 'Help'. Below the menu bar is a toolbar with icons for 'Back', 'Forward', 'Stop', 'Reload', 'Home', 'Search', and a star icon. The webpage content displays a form with the text 'How many pizzas would you like to order?' followed by a text input field containing the number '1'. Below this is a label 'Credit Card No' followed by a text input field. To the right of the 'Credit Card No' field is a button labeled 'Order'.

Security vulnerabilities of the GET method



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- The URL in the address bar of the browser can be
 - *https://www.deliver-me-pizza.com/confirm_order?session-id=3927a837e947df203784d309c8372b8e*



Security vulnerabilities of the GET method



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

- Before Alice further clicks “yes”, she copies this link and paste the link to her email to Meg. She asks, “shall we order this pizza?”.
- Meg can directly copy this link from the email and past it to the web browser to order the pizza without Alice’s consent
- The reason for this security vulnerability is that GET method leaks session-id

Security vulnerabilities of the GET method



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- The pizza delivery website may refer to other websites

```
<HTML>
<HEAD>
<TITLE>Pizza Order Complete</TITLE>
</HEAD>
<BODY>
Thank you for your pizza order. It will arrive piping hot within
30 to 45 minutes!
<A HREF=confirm_order?
session-id=3927a837e947df203784d309c8372b8e>
Click here to order one more pizza!
</A>
You may also be interested in trying our frozen pizzas at
<A HREF=http://www.grocery-store-site.com/>
GroceryStoreSite
</A>
</BODY>
</HTML>
```

POST Instead of GET

- Referers can leak through outlinks:
 - This `<a href="http://www.grocery-store-site.com/">` Link
 - Sends request:
 - *GET / HTTP/1.1 Referer: https://www.deliver-me-pizza.com/submit_order? session-id=3927a837e947df203784d309c8372b8e*
 - Session-id leaked to *grocery-store-site*'s logs!



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Benefits of POST



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY

- POST Request:
 - Session-id not visible in URL
 - Pasting into e-mail wouldn't leak it
 - Slightly inconvenient for user, but more secure
- Referers can still leak w/o user interaction
 - Instead of link, image:
 - `<a href=http://www.grocery-store-site.com/banner.gif>`
 - GET request for banner.gif still leaks session-id

```
POST /submit_order HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 45
session-id%3D3927a837e947df203784d309c8372b8e
```

Cookies

- Cookie - piece of state maintained by client
 - Server gives cookie to client
 - Client returns cookie to server in HTTP requests
 - session-id in cookie in lieu of hidden form field
 - HTTP/1.1 200 OK
 - Set-Cookie: session-id=3927a837e947df203784d309c8372b8e; secure
 - Secure dictates using SSL
 - Browser Replies:
 - GET /submit_order?pay=yes HTTP/1.1
 - Cookie: session-id=3927a837e947df203784d309c8372b8e



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Problems with Cookies



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Cookies are associated with browser
 - Sent back w/ each request
- If user doesn't log out, attacker can use same browser to impersonate user
- Session-ids should have limited lifetime

JavaScript

- Popular client-side scripting language
- Compute prices of an order:



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

```
Changes price submit.js x submit_order script changes.js x solution2 submit_order script.js x JavaScript (1).html x
1 <HTML> <HEAD> <TITLE>Order Pizza</TITLE> </HEAD> <BODY>
  <FORM ACTION="submit_order" METHOD="GET" NAME="f">
2 How many pizzas would you like to order?
3 <INPUT TYPE="text" NAME="qty" VALUE="1" onKeyUp="
  computePrice();">
4 <INPUT TYPE="hidden" NAME="price" VALUE="5.50"><BR>
5 <INPUT TYPE="submit" NAME="Order" VALUE="Pay">
6 <INPUT TYPE="submit" NAME="Cancel" VALUE="Cancel">
7 <SCRIPT>
8 function computePrice() {
9   f.price.value = 5.50 * f.qty.value;
10  f.Order.value = "Pay $" + f.price.value
11  }
12 </SCRIPT> </BODY> </HTML>
```

JavaScript



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Evil user can just delete JavaScript code, substitute desired parameters & submit!
 - Could also just submit request & bypass JavaScript
 - GET /submit_order?qty=1000&price=0&Order=Pay
- Warning: data validation or computations done by JavaScript cannot be trusted by server
 - Attacker may alter script in HTML code to modify computations
 - Must be redone on server to verify

Summary



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Web apps need to maintain state (HTTP stateless)
 - Hidden form fields
 - Cookies
 - Sessions
- Don't trust user input!
 - keep state on server (space-expensive)
 - sign transaction params (bandwidth-expensive)
 - Use cookies, be wary of cross-site attacks
 - No JavaScript for computations & trusted validations